

COMP 304: Project 2

Due: April 10th, 2026, 23:59

Mid Project Check: March 27th, 2026, 23:59

Instructor: Didem Unat

Notes: This assignment **must be** completed in **teams of 2**. You may discuss the problems with other students but the submitted work must be your own team's work. **Any material you use from web (or AI model) should be properly cited in your submission.** **Any sort of cheating** including directly copy-pasting AI generated code will be harshly PUNISHED.

This assignment is worth **19%** of your total grade (9.5% for the implementation and invited demos, and 9.5% for the project exam). The project exam will be conducted during the midterm exam. To be eligible to take the project exam, you must complete and submit the assignment before the deadline. **We strongly recommend that you start early.**

There will be a **mid-project check on March 27**, by which time we expect you to have completed Part I and started Part II in your Git repositories. You don't need to upload the project code to learn hub for the mid-project check. We will be using git classroom for the project.

Responsible TAs:

Name	Email	Office	Office Hours
Mohammad Issa	missa18@ku.edu.tr	ENG 230	Tuesdays 4:00-5:30 PM
Javid Baydamirli	jbaydamirli21@ku.edu.tr	ENG 230	Wednesdays 4:00-5:30 PM

Overview

This project focuses on **system call interception, process tracing, and kernel observability**. You will build a syscall sandbox system with three integrated components:

1. **eBPF Monitor** – Observes system calls made by specified processes
2. **Kernel Module** – Tracks process trees (parent and all descendant processes)
3. **ptrace Sandbox** – Intercepts and blocks specified system calls

Schedule

The project is divided into four parts, intended to be completed sequentially:

Week	Component	Purpose
1	eBPF Monitor	Syscall observation and logging
2	Kernel Module	Process tree tracking
3	ptrace Sandbox	Syscall interception and blocking
4	Integration	Connect all components

This ordering allows the eBPF monitor (Week 1) to be used as a debugging tool when implementing the ptrace sandbox (Week 3).

Codebase Structure

The starter code provides minimal skeletons that compile but need implementation:

```

syscall-sandbox/
+-- CMakeLists.txt           # Build system (provided)
+-- src/
|   +-- sandbox.c           # ptrace sandbox [IMPLEMENT]
|   +-- monitor.c           # eBPF monitor [IMPLEMENT]
|   +-- syscall_tester.c    # Test utility (provided)
|   +-- bpf_bridge.c        # eBPF <-> kernel bridge (provided)
+-- ebpf/
|   +-- Makefile            # eBPF build rules (provided)
|   +-- monitor.bpf.c       # eBPF program [IMPLEMENT]
+-- module/
|   +-- Makefile            # Kbuild Makefile (provided)
|   +-- syscall_module.c    # Kernel module [IMPLEMENT]
+-- challenges/
    +-- single-process.tar.gz # dialer, leaky (x86_64 + arm64)
    +-- multi-process.tar.gz  # hydra, mole (x86_64 + arm64)
    +-- x86_64-rootfs.tar.gz  # Chroot escape (x86_64)
    +-- aarch64-rootfs.tar.gz # Chroot escape (arm64)

```

Files marked with **[IMPLEMENT]** are minimal skeletons. They compile and run but do nothing useful – you must design and implement the functionality yourself.

Part I – eBPF Syscall Monitor (25 pts)

Build an eBPF program that observes system calls made by specific processes and reports them in real-time. This functions similarly to `strace` but runs inside the kernel with minimal overhead.

I-A: eBPF Program (15 pts)

Note: You will need to define appropriate BPF maps for the following

- Tracking which PIDs to monitor
- Tracking which syscall numbers to watch
- Sending events to userspace (ring buffer)

Create an eBPF program (`ebpf/monitor.bpf.c`) that:

1. Attaches to `tracepoint/raw_syscalls/sys_enter` (fires on every syscall)
2. Checks if the calling process is in your “watched list” (a BPF hash map)
3. Checks if this syscall is one you care about (another BPF hash map)
4. If both match → send an event to userspace via ring buffer

Sample Event Structure:

```
1 struct syscall_event {  
2     __u32 pid;  
3     __u32 syscall_nr;  
4     __u64 timestamp_ns;  
5     char comm[16];      // process name  
6 };
```

I-B: Userspace Loader (10 pts)

Extend `src/monitor.c` to:

1. Load the compiled eBPF program using `libbpf`
2. Pin the maps so other tools can update them:
 - `/sys/fs/bpf/tracked_pids`: which processes to monitor (PIDs)
 - `/sys/fs/bpf/watched_syscalls`: which syscalls to report (syscall NR)
3. Attach the program to the tracepoint
4. Poll the ring buffer and print events

By pinning both maps, the kernel module (Part II) can dynamically add/remove PIDs and syscalls, so your monitor will pick up changes **without restarting**.

Usage:

```
# Watch specific syscalls for a process
./monitor -w 39,41,257 -p 1234

# Output when PID 1234 makes those syscalls:
# [bash] pid=1234 syscall=39
# [bash] pid=1234 syscall=257
```

Testing with syscall_tester:

```
# Terminal 1: Start tester (prints its PID)
./syscall_tester 39          # calls getpid every second

# Terminal 2: Monitor it
./monitor -w 39 -p <tester_pid>
```

Files to implement:

- ebpf/monitor.bpf.c: eBPF program (maps, tracepoint handler)
- src/monitor.c: Userspace loader (load, attach, poll events)

Part II – Kernel Module (25 pts)

Build a kernel module that tracks process trees. When a program forks children, and those children fork grandchildren, the module automatically tracks all descendants.

II-A: Basic Configuration Interface (10 pts)

Create `/proc/syscall_sandbox` with:

- **Write `+NNN`:** Add syscall number `NNN` to a “watch list”
- **Write `-NNN`:** Remove syscall number from watch list
- **Write `clear`:** Clear the watch list
- **Read:** Show current configuration

```
$ echo "+257" | sudo tee /proc/syscall_sandbox
+257
$ echo "+41" | sudo tee /proc/syscall_sandbox
+41
$ echo "register 1234" | sudo tee /proc/syscall_sandbox
register 1234
$ cat /proc/syscall_sandbox
BPF map: not attached
Tracked PIDs (1):
 1234
Blocked syscalls (2):
 257
 41
```

II-B: Process Tree Tracking (15 pts)

Automatically track when watched processes create children.

1. **Connect to eBPF monitor:** Accept BPF map file descriptors via:

```
echo "setmap_pids <fd>" | sudo tee /proc/syscall_sandbox
echo "setmap_syscalls <fd>" | sudo tee /proc/syscall_sandbox
```

This gives the kernel module direct access to the eBPF maps. The module can then:

- Update `tracked_pids` when processes fork/exit
- Update `watched_syscalls` when you add/remove syscalls via `+N/-N`

Your eBPF monitor picks up all changes **without needing to restart**.

2. **Automatic fork tracking:** Use `tracepoint_probe_register()` on `sched_process_fork`. When a tracked process forks:

- Add the child PID to your internal list
- Add the child PID to the eBPF `tracked_pids` map

3. **Automatic cleanup:** Use `sched_process_exit` to remove exited processes from both the internal list and the BPF map.

The **bpf-bridge** Tool:

A helper tool `bpf-bridge` (provided in `src/bpf_bridge.c`) handles passing the BPF map file descriptors to the kernel module. It opens both pinned maps (`/sys/fs/bpf/tracked_pids` and `/sys/fs/bpf/watched_syscalls`), writes the `setmap_*` commands to `/proc/syscall_sandbox`, and keeps running to hold the file descriptors open.

Files to implement:

- `module/syscall_module.c`: Kernel module (/proc interface, process tracking, BPF map integration)

Part III – ptrace Sandbox (30 pts)

Implement syscall blocking using ptrace. This component intercepts system calls and either blocks them (returning `-ENOSYS`) or terminates the process.

What is a Sandbox?

A sandbox is a restricted execution environment. The process inside thinks it's running normally, but certain operations are blocked. For example:

- Block `socket()` → process can't access the network
- Block `execve()` → process can't run other programs
- Block `openat()` → process can't open files

We'll use **ptrace** to intercept syscalls and optionally block them.

III-A: Single-Process Sandbox (20 pts)

Extend `src/sandbox.c` to:

1. **Fork a child** that calls `PTRACE_TRACEME` and executes the target program
2. **Intercept every syscall** using `PTRACE_SYSCALL`
3. **Block forbidden syscalls** by modifying registers (makes syscall return `-ENOSYS`)
4. **Report blocked attempts** to `stderr`
5. **Support kill mode** that terminates the process instead of just failing the syscall

Usage:

```
# Block openat syscall
./sandbox -b 257 -- /bin/cat /etc/hostname
# [sandbox] Blocked syscall 257 (openat)
# cat: /etc/hostname: Function not implemented

# Kill mode: terminate on violation
./sandbox -k -b 257 -- /bin/cat /etc/hostname
# [sandbox] KILLED (syscall 257)
```

III-B: Multi-Process Sandbox (10 pts)

Extend the sandbox to track child processes:

1. Set `PTRACE_O_TRACEFORK` | `PTRACE_O_TRACEVFORK` | `PTRACE_O_TRACECLONE`
2. Handle `PTRACE_EVENT_FORK` / `PTRACE_EVENT_CLONE` events
3. Track all descendants and apply the same blocking rules

Verification:

```
# Single-process challenges
./sandbox -b <syscall> -- ./challenges/single-process/dialer

# Multi-process challenges (require -c flag)
./sandbox -c -b <syscall> -- ./challenges/multi-process/hydra
```

Files to implement:

- `src/sandbox.c`: ptrace sandbox (fork, trace, intercept, block)

Part IV – Challenges (20 pts)

Analyze and block malicious system calls using your complete sandbox system.

Running the Full System

Wire all three components together:

1. Start eBPF monitor (creates and pins both maps)
2. Load kernel module
3. Run bpf-bridge to connect the maps to the kernel module
4. Configure syscalls via /proc – the monitor sees changes live
5. Run ptrace sandbox with fork tracking

```
# Terminal 1: Start eBPF monitor
./monitor

# Terminal 2: Load kernel module and start bridge
sudo insmod module/syscall_module.ko
sudo ./bpf-bridge

# Terminal 3: Configure and run
echo "+41" | sudo tee /proc/syscall_sandbox    # watch socket
echo "+257" | sudo tee /proc/syscall_sandbox   # watch openat
echo "register $$" | sudo tee /proc/syscall_sandbox
./sandbox -c -b 41 -b 257 -- ./challenges/multi-process/hydra
```

The kernel module updates the BPF maps when you add syscalls or when processes fork. All three components work together: eBPF monitor shows syscalls in real-time, ptrace sandbox blocks them, and /proc/syscall_sandbox shows the configuration.

Challenge Binaries

The challenge binaries are provided in the challenges/ directory. Extract them first:

```
cd challenges
tar -xzf single-process.tar.gz
tar -xzf multi-process.tar.gz
```

For the chroot escape challenge, extract the appropriate rootfs for your architecture:

```
# x86_64
tar -xzf x86_64-rootfs.tar.gz
# OR aarch64 (might be needed for those of you running on Macs)
tar -xzf aarch64-rootfs.tar.gz
cd x86_64-rootfs # or aarch64-rootfs
sudo chroot . /bin/busybox ash
# / ls / # feel free to explore the environment inside the chroot
# ./breakout # (the binary that escapes the chroot to gain root access on your machine)
```

Analyze each binary to identify suspicious syscalls and block them:

Challenge	Points	Description
dialer	3	Single-process challenge
leaky	2	Single-process challenge
hydra	3	Multi-process challenge (requires <code>-c</code> flag)
mole	3	Multi-process challenge (requires <code>-c</code> flag)
Chroot escape	4	Chroot breakout challenge

Required documentation (in your README):

- For each challenge, show screenshots of the **full system working together**:
 - Terminal showing eBPF monitor catching syscalls (before and after sandboxing)
 - Terminal showing ptrace sandbox blocking syscalls (after sandboxing)
 - Output of `cat /proc/syscall_sandbox` showing configuration (before sandboxing, or when something interesting happens (forks))
- Explain which syscall(s) you identified as problematic and why

Building

Requirements

- **Linux kernel 5.8+** – Required for BPF ring buffers. Check with `uname -r`.

Prerequisites

```
# Ubuntu/Debian
sudo apt install build-essential cmake linux-headers-$(uname -r) \
    libbpf-dev bpftool

# Fedora
sudo dnf install @c-development cmake kernel-devel libbpf-devel \
    strace bpftool
```

Build Commands

```
# STEP 1: Generate vmlinux.h and build eBPF program
cd ebpf && make

# STEP 2: Build userspace tools (sandbox, monitor, bpf-bridge)
mkdir build && cd build
cmake ..
make

# STEP 3: Build kernel module
cd module && make
```

Common Syscall Numbers (x86_64)

Number	Name	Description
0	read	Read from file descriptor
1	write	Write to file descriptor
39	getpid	Get process ID
41	socket	Create socket
56	clone	Create child process
57	fork	Create child process
59	execve	Execute program
257	openat	Open file

Full list: `ausyscall --dump` or `/usr/include/asm/unistd_64.h`

Point Distribution

Component	Points
Part I – eBPF Monitor	25
Part II – Kernel Module	25
Part III – ptrace Sandbox	30
Part IV – Challenges	20
Total	100

Deliverables

Submission Guidelines

Please submit your assignment to LearnHub as a single zip file named `username1_username2.zip`.

Your submission must include:

- **Your Source Code:** A copy of the latest cloned version of your Git repository.
- **A README File:** This must include a link to your GitHub repository, a detailed description of your implementation, and your challenge binary analysis (including screenshots).

Your submission must NOT include:

- Do **not** include any executable files or object files (`.o`).
- Do **not** include the challenge binaries or solution files.

GitHub Workflow Requirements:

- **Repository Setup:** Join the Github Classroom at <https://classroom.github.com/a/nyiCfx6J> and make sure there is a repository for your group.
- **Commit History:** You must make consistent, incremental commits from start to finish so we can track your progress. Do not commit the entire assignment at once at the very end; doing so may result in a grading penalty. We will review your commits during the assignment evaluation.

General Rules & Grading:

- **Environment:** You are required to implement the assignment on a Linux environment.
- **Interviews:** Grading will take place via an **in-person interview**. Both team members must be fully knowledgeable about the entire implementation. If you cannot adequately explain your code or demonstrate a clear understanding of the concepts, **you may receive zero** for the corresponding part of the project.

References

- <https://elixir.bootlin.com/linux/latest/source> (Linux Kernel)
- <https://eli.thegreenplace.net/2011/01/23/how-debuggers-work-part-1> (ptrace tutorial series (by example of debuggers))
- <https://www.linuxjournal.com/article/6100> (ptrace fundamentals)
- <https://sysprog21.github.io/lkmpg/#the-proc-file-system> (kernel module book/guides, good for /proc resources too)
- <https://docs.cilium.io/en/stable/bpf/> (useful for bpf maps and object pinning)
- <https://nakryiko.com/posts/libbpf-bootstrap/> (useful for a bpf usage introduction)
- <https://github.com/iovisor/bcc/tree/master/libbpf-tools> (bpf example tools)
- <https://github.com/earthquake/chw00t> (other examples of chroot breaking methods).
- <https://brennan.io/2016/10/13/kernel-dev-ep1/> (an approachable kernel devel experience)
- `man 5 proc`
- `man 2 ptrace`
- `man 2 chroot`

GOOD LUCK.